The set model on a slice

Test 0 of docs/23: the two circularities of Q2, and a well-founded model for a slice of Ochr

6 November 2026 · a paper result, with a Lean encoding of part of it · ochr/docs/24

Summary. The set model exists on the slice. The slice is `Nat`; a type function defined by recursion (`Cells`); a type variable $A : \texttt{Type}_0$ with the injectivity relation R at it; a function type returning a borrow; and a Π into `Prop` over $\texttt{Type}_0$. docs/18′s Q2 gave two reasons why the model as written was not yet a definition, and neither is an obstacle. First, the meaning of a type and R were defined in terms of each other. If the meaning of a function type is taken to be its *good* functions only, R at every type is the meaning itself. No pairing of a type with a set of good elements is needed, and the Lean encoding shows that pairing would in fact break conversion of functions. Second, "induction on the type" had nothing smaller to recurse on. The meaning is instead defined on *terms*, by recursion on the position of an item in the program, the structure of the term, and, inside a recursive function, the rank of its decreasing argument. A type computed by code then means what its code computes, and a type variable means what the environment gives it. Universe levels matter only for sizes, not for well-foundedness. One consequence changes docs/23 and docs/18: a recursive function has a meaning whether or not it passed [Rec], so [Rec] belongs to membership, not to well-formedness. Not covered: the meaning of the state (borrows, loans, holes, moved-out places) and therefore of `Id`, which test 2 must add (Section 10).

Contents

1 What this test is 2
2 The two circularities, and why they go 2
2.1 Meaning and R, defined in terms of each other 2
2.2 "Induction on the type" has nothing smaller 3
3 The model 4
3.1 Universes, the proof, and the error value 4
3.2 Environments and assignments 4
3.3 Meanings of terms and values 5
3.4 Inductive declarations 5
3.5 Function types 6
3.6 Recursive functions: guarded recursion 7
4 Well-foundedness 8
5 The slice, worked 8
5.1 `Nat` 8
5.2 A type function defined by recursion: `Cells` 8
5.3 A type variable, with R at it 9
5.4 A function type returning a borrow 9
5.5 A Π into `Prop` over $\texttt{Type}_0$ 10
6 Sizes: where universe levels matter 10
7 The Lean encoding 10
8 The choices 11
9 What this test does not settle 12
10 What test 2 inherits 12
11 Summary for docs/18 13

1 What this test is

docs/23 chose to prove Ochr consistent by meaning preservation in the set model, and named the model itself as the main risk, shared with the superseded route. docs/18′s open question Q2 says that the model as first written was not a valid definition, for two reasons, restated in Section 2. Test 0 asks whether a well-founded definition exists, on a slice chosen to contain each feature Q2 worries about:

- `Nat`;
- a type function defined by recursion on data, `Cells`;
- a type variable $A : \texttt{Type}_0$, with the injectivity relation R at it;
- a function type returning a borrow, whose backward functions must be injective;
- a Π into `Prop` over $\texttt{Type}_0$ (impredicativity).

Test 2 builds on this document, so two things are defined in general rather than for the slice only: function types (borrow parameters, a borrow result, function-typed parameters, and R), and the meaning of type-level code (calls of type functions) with its well-founded measure.

What "exists" means here. A definition, in ZFC with inaccessible cardinals, of two things:

- a meaning for every term and value of the slice: a set, or a distinguished *error value*;
- for every type, a set.

The definition must be well founded: no clause refers, directly or through others, to the meaning it is defining.

Existence does *not* include *membership*: that the meaning of an accepted definition lies in the meaning of its type. Membership is the soundness theorem's job (docs/23, point 5, and docs/18′s central theorem), and test 2 starts on it.

Result. The model exists on the slice, paper argument, with a Lean encoding of most of it (Section 7). Neither circularity is an obstacle; both came from how the model was first phrased (Section 2). The definition needed eleven choices, named in Section 8. One of them changes docs/23 and docs/18 (Section 3.6). Nothing in the slice failed. What is not covered is listed in Section 9; the next hardest piece is the meaning of the state, which `Id` needs inside the meaning of types (Section 10).

> **Reminder.** Notation from docs/18. $[\![x]\!]_\gamma$ is the meaning of x under an assignment γ of the unknowns. An *abstract value* σ is an unknown introduced by the checker, with type $\Delta(\sigma)$. A Π-type's *class* is read off its codomain term: it returns types, returns proofs, or neither ("other"). [Rec] is the recursion check: every recursive call passes, at the declared decreasing position, a strict subterm of that parameter's entry value. docs/18′s conditions on declarations, used below: (I1) strict positivity: the declared type `D` occurs in a field type only as the whole field type, or nested at a *nestable* parameter of an earlier inductive, never left of a Π, under `&`, or as the argument of a type function; (I2) the arguments of an occurrence of `D` never mention `D`; (I3) a parameter is never called in a field type; (I4) parameter types do not mention `D`; (I5) fields form a telescope; (I6) every field of an inductive in $\texttt{Type}_0$ or `Prop` lives in `Prop` or $\texttt{Type}_0$; (I7) a match on a proof produces data or a type only from a subsingleton (no constructors, or one whose fields are all proofs); (I8) recursion is never on a proof.

2 The two circularities, and why they go

2.1 Meaning and R, defined in terms of each other

docs/18 interprets a function type that returns a borrow as a subset R of a set of functions, namely those whose backward functions are injective. It defines $R_A \subseteq [\![A]\!]$, for each type A, by induction on A, and at a function type it reads R at the parameter and result types. So the meaning of a function type needs R, and R needs meanings.

The two need not be separate. The meaning of a function type can be defined directly as its good part. For a function type returning a borrow:

$$\left[\!\left[\Pi\big(\overline{x} : \overline{A}\big).\&U\right]\!\right] = \left\{ F \in \prod_{\overline{d} \in \mathrm{Dom}} \left([\![U]\!] \times ([\![U]\!] \to \mathrm{Fin})\right) \;\middle|\; \mathrm{back}_F\big(\overline{d}\big) \text{ is injective for every } \overline{d} \right\},$$

where the domain Dom is built from the *meanings* of the parameter types (Section 3.5). Every argument the clause quantifies over is then already good, because it ranges over the meaning of its parameter type, which is a good set by the same definition. docs/18′s clauses for R, one by one:

docs/18′s clause for R	**in the merged definition**
Propositions and universes: $R_A = [\![A]\!]$.	Unchanged.
Inductive: R holds of a constructor value when it holds of each field.	Automatic. The least fixed point is built from the fields' meanings, which are good sets.
$R_{\&T} = R_T$.	Automatic. A borrow parameter ranges over $[\![T]\!]$.
Function type: for all arguments in R, the forward value is in R and the backward function is injective.	This *is* the function-type clause: arguments range over the domains' meanings, the forward value over the result's.

So $R_A = [\![A]\!]$ at every type, and R disappears as a separate object.

Type variables. This is where docs/18 saw the need for pairs: a type as a set together with its subset of good elements, and a universe as a set of such pairs, because "R at a type variable is undefined". In the merged definition, a type variable $A : \mathsf{Type}_0$ means whatever the environment gives it, an arbitrary set in V_{κ_0}, and R at A is the whole of that set. This is sound if no argument ever needs the elements of that set to be good beyond belonging to it.

An Ochr program can use a value $x : A$ as a function only after A has become a Π-type. In this route that can happen in exactly one way: by transport along a proof $h : \mathsf{Eq}(\mathsf{Type}_0, A, \Pi(...)...)$. The type of h means $\{\bullet\}$ only when the set A stands for *is* the meaning of that Π-type, which is a good set. Under any other choice the hypothesis is false and there is nothing to prove. Conversion cannot do it either: there is no second path that instantiates the unknown, and the checker's conversion never identifies an unknown type with a Π-type.

A type computed by code is no exception. Take $F(b)$ where one arm of F returns a Π-type: it means $[\![F]\!]$ at the meaning of b, which in that arm is the good set, because every clause that produces a type produces a good set.

Why: R exists to throw out functions whose backward function ignores its argument, which the checker's reasoning about `Id` *rules out (docs/18′s example* `PF`*). Putting the condition in the function-type clause throws them out at the only place they could come from. A universe of pairs would hold sets with bad elements, and nothing in Ochr can reach them.*

Pairing is not just unnecessary but harmful. The Lean encoding (Section 7) first paired a carrier with a predicate for R, and found two functions out of the meaning of `TailM`'s type, `g0` and `g1`, that agree on every good argument yet are distinct. The file keeps this counterexample as `Paired.agree_on_good` and `Paired.differ`. They differ only at a *junk* argument: a borrow-returning function whose backward function is constant. Ochr's conversion compares two functions at generic arguments, and an assignment gives those only good values, so it identifies functions that agree on good arguments ([Conv-fun]). With pairs, convertible functions would have different meanings and conversion would be unsound. The repair would be a partial equivalence relation on each carrier. With carriers that hold only good elements, the two functions are equal (the same pair, as `f0` and `f1`, gives `f0_eq_f1`), and [Conv-fun] is function extensionality (`pi_ext`, `brwRet_ext`). That settles choice C1 below.

2.2 "Induction on the type" has nothing smaller

docs/18′s second reason: a function type's codomain is computed by running code at each argument, so it is not a subterm of the type; and a type variable has no structure. Both are true of *types as values*. Both stop mattering when the recursion is on *terms*. A type is a term whose meaning is a set, and its meaning is defined like the meaning of any other term:

- **Π-types.** A Π-type *term* $\Pi(x : A).B$ has its codomain B as a subterm, even though the Π-type *value* computes B by running code at each argument. The value is a closure $\langle \kappa \vdash \Pi(x : A).B \rangle$, and it means what the term means, with the captured values' meanings for its captures. So $[\![\Pi(x : A).B]\!]$ refers to the meanings of A, and of B at each argument: both subterms.

- **Computed types.** A computed type $F(a)$, for an earlier global F, means $\mathrm{app}([\![F]\!], [\![a]\!])$, where $[\![F]\!]$ is the meaning of an earlier item of the program: a function that already exists. Nothing at the use site recurses into F's body.
- **Recursive type functions.** F's own meaning is defined once, at F's place in the program, from its body. It is defined by recursion on the rank of its decreasing argument, guarded so that a call that does not decrease means the error value (Section 3.6).
- **Type variables.** A type variable means what the environment gives it.

So the definition recurses on three things, lexicographically: the position of the item in the program, the structure of the term, and, inside a recursive function, the rank of its decreasing argument. All three are well founded (Theorem 5).

Where a regress would come from. docs/23 recorded the worry that interpreting code that returns Type_i needs the meaning of Type_i, and so on. That would be a regress if $[\![\mathsf{Type}_i]\!]$ were defined as the set of meanings of the level-i types: a universe of *codes*, defined together with its decoding (an inductive–recursive definition). Set theory does not need one. $[\![\mathsf{Type}_i]\!] = V_{\kappa_i}$ is given outright, as a level of the cumulative hierarchy, and code that returns a type returns some member of it. The Lean encoding confirms this: Lean has no induction–recursion, and none was needed, because Type_0 is read as Lean's `Type`, every type of level 0 (Section 7).

Universe levels are needed only for *sizes*: that a type at Type_i means a member of V_{κ_i} (Section 6).

> **Quick explainer: well-founded recursion and rank.** A definition by *well-founded recursion* defines $f(x)$ using f only at arguments below x in a relation that has no infinite descending chain. Such a definition has exactly one solution (the recursion theorem). The *rank* of a set is the stage of the cumulative hierarchy at which it first appears: $\mathrm{rank}(s) = \sup\{\mathrm{rank}(e) + 1 \mid e \in s\}$. Every element of a set has smaller rank than the set, so a component of a tuple has smaller rank than the tuple. Ranks are ordinals, and ordinals have no infinite descending chain.

3 The model

3.1 Universes, the proof, and the error value

- **Universes.** Fix inaccessible cardinals $\kappa_0 < ... < \kappa_N$, one for each Type_i the program writes. Then $[\![\mathsf{Type}_i]\!] = V_{\kappa_i}$ and $[\![\mathsf{Prop}]\!] = \{\emptyset, \{\bullet\}\}$, where $\bullet$ is a fixed object, "the proof".
- **Where meanings live.** Every meaning is a member of $U = V_{\kappa_N+\omega}$. Since $\kappa_N + \omega$ is a limit ordinal, U is closed under pairing, subsets, function sets between its members, and products indexed by its members, and it contains V_{κ_N}.
- **The error value.** err is a fixed object not in U. It is the meaning of a term that goes wrong: applying something that is not a function, matching a value against arms none of which fits, calling a recursive function at an argument that does not decrease (Section 3.6). Every clause below is *strict*: unless it says otherwise, its result is err if a meaning it needs is err or has the wrong shape.
- **Tags.** Each constructor C of each inductive gets a tag $\ulcorner \mathsf{C} \urcorner \in V_\omega$, distinct from every other constructor's.

3.2 Environments and assignments

An *environment* η maps variables and abstract values to members of U. We write $[\![t]\!]_\eta$ for the meaning of a term t in η. For the abstract values of a state, an environment is docs/18′s *assignment* γ. It gives the abstract values $\sigma_1, ..., \sigma_m$, in the order they were created, elements $\gamma(\sigma_i) \in [\![\Delta(\sigma_i)]\!]_\gamma$. Since $\Delta(\sigma_i)$ mentions only abstract values created before σ_i, each condition is about a set that is already determined. This is the sense in which unknowns range over arbitrary set elements: $\gamma(\sigma)$ may be any member of the set, including a function no program computes. docs/18′s definition of an assignment also asks that $\gamma(\sigma)$ be "in R at σ's type". By Section 2.1 that condition is now the same as $\gamma(\sigma) \in [\![\Delta(\sigma)]\!]_\gamma$, so it can be dropped.

3.3 Meanings of terms and values

The meaning is defined one item of the program at a time. Before item k, the *signature meaning* M gives each earlier definition g a meaning $M(g) \in U \cup \{\text{err}\}$, and each earlier inductive D a family $M(\mathsf{D})$ (Section 3.4). We write $\text{app}\big(F, \overline{d}\big)$ for $F\big(\overline{d}\big)$ when F is a function whose domain contains $\overline{d}$, and for err otherwise. Given M, the meaning of a term is defined by the clauses below. They cover the pure terms of the slice; the clauses for the state are test 2′s (Section 10).

Term	Meaning in η
a variable x, an abstract value σ	$\eta(x)$, $\eta(\sigma)$
Type_i, `Prop`	V_{κ_i}, $\{\emptyset, \{\bullet\}\}$
an inductive type $\mathsf{D}(\overline{a})$, `Nat` included	$M(\mathsf{D})(\overline{e})$, where $\overline{e}$ are the meanings of $\overline{a}$, when $\overline{e}$ is in D's parameter domain
a constructor $\mathsf{C}(\overline{a}; \overline{t})$	$(\ulcorner \mathsf{C} \urcorner, e_1, ..., e_n)$, where e_i is the meaning of t_i. The parameters $\overline{a}$ are not read (docs/18, (V2)).
a call $g(\overline{t})$ of a global	$\text{app}(M(g), \overline{e})$, where $\overline{e}$ are the meanings of $\overline{t}$
a call $x(\overline{t})$ of a variable	$\text{app}(\eta(x), \overline{e})$
match p $\{\mathsf{C}_i(\overline{y}_i) \Rightarrow t_i\}$ on data	the meaning of t_i in η extended with $\overline{y}_i \mapsto \overline{v}$, when p means $(\ulcorner \mathsf{C}_i \urcorner, \overline{v})$; err when no arm fits
a match on a proof that produces data or a type	the inductive is a subsingleton (I7): with one constructor, the meaning of its arm with every field bound to $\bullet$; with none, err
any term whose declared sort is `Prop` (a proof)	$\bullet$, whatever its structure
$\mathsf{Eq}(A, a, b)$	$\{\bullet\}$ if a and b have the same meaning, $\emptyset$ otherwise. A's meaning must not be err, but is not otherwise read.
$\mathsf{J}(A, a, b, P, h, t)$, **rewrite** h **in** t, at data	the meaning of t
$\Pi\big(\overline{x} : \overline{A}\big).B$	Section 3.5
fix $f\big(\overline{x} : \overline{A}\big) : B$ **by** $x_j \coloneqq t$, and λ	Section 3.6
let $x = a; b$, pure	the meaning of b in η extended with $x \mapsto$ the meaning of a
a sealed program $\lceil t \rceil$	the meaning of t, a closed term

Values mean what the terms they come from mean:

- a constructor value means its tag and its fields' meanings;
- $\star$ means $\bullet$;
- a closure $\langle \kappa \vdash c \rangle$ means the meaning of its code c in the environment that gives its captures κ their meanings, and so does a Π-closure;
- a global's name means $M(g)$;
- a type value means the meaning of the type term it is;
- an abstract value means $\gamma(\sigma)$;
- a sealed program means its program's meaning.

Borrows, loans and $\perp$ are test 2′s (Section 10).

Why: data is read untyped*: a constructor value means a tagged tuple whatever type it is used at, which is what lets the step lemma speak of values before their types are known to be right. Functions are read on the domain their annotations give. A function must have some domain, and the annotation is the only one available without typing the body.*

Proofs are read by their declared sort. A proof means $\bullet$ because its declared sort is `Prop`, which is syntactic (D55), not because of anything in its structure. This is forced by impredicativity. A proof of $\Pi(x : A).P$ must mean $\bullet$, not a function, so applying a proof cannot be function application, and the clause cannot look inside.

3.4 Inductive declarations

Take a declaration **inductive** $\mathsf{D}\ \big(\overline{x} : \overline{X}\big) : s \coloneqq \mathsf{C}_1\big(\overline{g}_1 : \overline{B}_1\big) \mid ...$ with $s \in \{\mathsf{Prop}, \mathsf{Type}_0\}$.

- Its *parameter domain* P_D is the set of tuples $\overline{a}$ with each a_i in the meaning of X_i at $\overline{a}_{<i}$.

- A *family* is a function $X : P_{\mathsf{D}} \to \mathcal{P}\big(V_{\kappa_0}\big)$. Families form a complete lattice under pointwise inclusion; it is a set, since P_{D} and $\mathcal{P}\big(V_{\kappa_0}\big)$ are in U.
- The operator Φ sends a family X to the family

$$\Phi(X)(\overline{a}) = V_{\kappa_0} \cap \bigcup_i \left\{ (\ulcorner \mathsf{C}_i \urcorner, e_1, ..., e_k) \mid e_j \in [\![B_{i,j}]\!]^X_{\eta_j} \right\},$$

where η_j binds the parameters to $\overline{a}$ and the earlier fields to $e_1, ..., e_{j-1}$. In $[\![\cdot]\!]^X$, every occurrence $\mathsf{D}\big(\overline{b}\big)$ in a field type is read as X at the meaning of $\overline{b}$.

Lemma 1 (the operator is monotone). Under (I1)–(I5), if $X \subseteq Y$ pointwise then $\Phi(X) \subseteq \Phi(Y)$.

Proof sketch. By induction on the field type, show that its meaning grows with X.

- A whole-field occurrence of D reads X directly.
- A nested occurrence, at a nestable parameter of an earlier inductive E, reads $M(\mathsf{E})$ at a set that grows with X. $M(\mathsf{E})$ is monotone in its nestable parameters: it is the least fixed point of an operator monotone jointly in the family and in those parameters (this lemma for E, by induction on the program), and such a least fixed point is monotone in the parameter.
- Nothing else depends on X: the arguments of an occurrence never mention D (I2), parameters are never called (I3), and D is never the argument of a type function (I1).
- An equation field $\mathsf{Eq}(T, a, b)$ reads only the meanings of a and b, which are earlier fields' values, not T. So an equation over D itself, as Fording writes, is constant in X. (This is why the Eq clause does not read its type, choice C10.)
- Fields form a telescope (I5). A Σ over a growing index set of growing fibres grows.

□

Definition 2 (meaning of an inductive). For $s = \mathsf{Type}_0$, $M(\mathsf{D})$ is the least fixed point of Φ, which exists by the Knaster–Tarski theorem. For $s = \mathsf{Prop}$, it is that least fixed point *truncated*: $\{\bullet\}$ at $\overline{a}$ if the fixed point is non-empty there, $\emptyset$ otherwise. Truncation is applied after the fixed point, so while the fixed point is built, a recursive field of a proposition ranges over the untruncated family, as in Werner's model.

The intersection with V_{κ_0} makes the lattice a set without first proving sizes. Section 6 shows that it removes nothing, by (I6).

> **Quick explainer: least fixed points by Knaster–Tarski.** In a complete lattice (every subset has a least upper bound and a greatest lower bound), every monotone map Φ has a least fixed point: the meet of all X with $\Phi(X) \leq X$. Unlike iterating Φ from the bottom ω times, this needs no continuity, so it does not matter how many stages the construction would take.

3.5 Function types

Take a Π-type term $\Pi(x_1 : A_1, ..., x_n : A_n).B$. Its class and whether it returns a borrow are read off the codomain term, as the checker reads them. Let $I = \{i \mid A_i = \&T_i\}$ be its borrow parameters, and let $A_i^\circ = T_i$ for $i \in I$ and A_i otherwise. In η:

- Dom is the set of tuples $\overline{d}$ with each d_i in the meaning of A_i°, in η extended with $\overline{x}_{<i} \mapsto \overline{d}_{<i}$. A borrow parameter contributes its *starting content*, and later parameter types read it as $* x_i$.
- $\mathrm{Fin}\big(\overline{d}\big) = \prod_{i \in I} T_i\big(\overline{d}\big)$ is the set of tuples of *final contents*, where $T_i\big(\overline{d}\big)$ is the meaning of T_i at $\overline{d}_{<i}$.
- $B\big(\overline{d}\big)$ is the meaning of B in η extended with $\overline{x} \mapsto \overline{d}$: the codomain read at the call's start, as [Call-type] reads it.

The meaning is given by one of four clauses:

- **(Π-proof)** Class "returns proofs": $\{\bullet\}$ if $B\big(\overline{d}\big) = \{\bullet\}$ for every $\overline{d} \in \mathrm{Dom}$; $\emptyset$ if every $B\big(\overline{d}\big)$ is a truth value and some is $\emptyset$; err otherwise.
- **(Π-type)** Class "returns types": $\prod_{\overline{d} \in \mathrm{Dom}} B\big(\overline{d}\big)$. There are no final contents: a call of a type-returning function is erased, so it runs on a private copy and its writes never reach the caller (RULES P2).
- **(Π-data)** Class "other", B borrow-free: $\prod_{\overline{d} \in \mathrm{Dom}} \big(B\big(\overline{d}\big) \times \mathrm{Fin}\big(\overline{d}\big)\big)$.

- **(Π-borrow)** Class "other", $B = \&U$: the functions $F \in \prod_{\overline{d} \in \mathrm{Dom}} \big(U\big(\overline{d}\big) \times \big(U\big(\overline{d}\big) \to \mathrm{Fin}\big(\overline{d}\big)\big)\big)$ with $\mathrm{back}_F\big(\overline{d}\big)$ injective for every $\overline{d}$. Here $F\big(\overline{d}\big) = \big(\mathrm{fwd}_F\big(\overline{d}\big), \mathrm{back}_F\big(\overline{d}\big)\big)$, and $U\big(\overline{d}\big)$ is the meaning of U at $\overline{d}$. There is at least one borrow parameter (D44).

Some cases:

- A function-typed parameter, $A_i = \Pi(...)...$, contributes its meaning by these same clauses: a good set.
- A borrowed function, $\&(\Pi(...)...)$, allowed since D66, contributes that meaning as both its starting and its final content.
- If a later parameter type is false, Dom is empty, and the meaning is the set holding only the empty function.
- Each clause uses only the meanings of A_i, T_i, U and B, which are subterms of the Π-term.

Why: the shape follows Aeneas. A borrow passed in is a value in and a value out. A borrow returned is a forward value, and a backward function from what the caller leaves in the borrow to what the lenders get back. Only injectivity is required, not the laws of a lens: `TailM`'s backward function $v \mapsto n + v$ does not give back v when read again, and is still an Ochr function (docs/17, decision 5).

3.6 Recursive functions: guarded recursion

Take **fix** $f\big(\overline{x} : \overline{A}\big) : B$ **by** $x_j \coloneqq t$ in η. A global's body is this with η empty and the signature meaning before it. Write Dom for its domain (Section 3.5). For $x_j : \&\ \mathsf{D}$, the decreasing argument d_j is the starting content.

Definition 3 (meaning of a recursive function). $[\![\textbf{fix } f...]\!]_\eta$ is the unique function F on Dom with

$$F\big(\overline{d}\big) = [\![t]\!]_{\eta\left[\overline{x} \mapsto \overline{d},\, f \mapsto F \restriction\limits_{\overline{d}}\right]},$$

where the *guarded restriction* $F \restriction\limits_{\overline{d}}$ is the function on Dom that sends $\overline{e}$ to $F(\overline{e})$ if $\mathrm{rank}\big(e_j\big) < \mathrm{rank}\big(d_j\big)$, and to err otherwise. A `fix` without `by` (a λ) means $\overline{d} \mapsto [\![t]\!]_{\eta[\overline{x} \mapsto \overline{d}]}$, and f is not in scope in it (D33).

It is well defined. The right-hand side reads F only at arguments whose j-th component has smaller rank. That relation on Dom is well founded, so by the recursion theorem exactly one F satisfies the equation. Nothing about the body is needed: the definition exists whether or not the body passed [Rec].

Consequence for docs/23 and docs/18. Both put "every closure and global in the state has passed [Rec]" in the well-formedness invariant, on the ground that a `fix` has a meaning only through well-founded recursion. With the guard, every `fix` has a meaning, checked or not. [Rec] instead ensures that the guard never fires, which is part of membership.

Lemma 4 (the defining equation). Suppose that at every $\overline{d} \in \mathrm{Dom}$, the meaning of t uses the value of f only at arguments $\overline{e}$ with $\mathrm{rank}\big(e_j\big) < \mathrm{rank}\big(d_j\big)$. Then $F\big(\overline{d}\big)$ equals the meaning of t with $f \mapsto F$ itself.

Proof sketch. F and $F \restriction\limits_{\overline{d}}$ agree at every argument the body uses. □

The hypothesis is the semantic form of [Rec]. [Rec] gives it in the following way.

- An accepted recursive call passes, at position j, a strict subterm of the entry value as refined.
- Under an assignment that satisfies the path's refinements (docs/23, point 5), that argument's meaning is a component, at some depth, of the tuple that is the entry value's meaning.
- A component has smaller rank.
- A closure formed in the body that calls f captures $F \restriction\limits_{\overline{d}}$. It is checked against the same entry value (R4), so it too calls f only below.
- [Rec] forbids every other use of f: the name occurs only as the head of a call.
- (I8) excludes recursion on a proof. A proof means •, and its rank never decreases, so the guard would fire.

The Lean encoding proves this lemma for type functions recursive on `Nat` (Section 7, `tfun_eq`).

The third historical closed proof of `False` (docs/18, the recursive local function that captured a value it split on) is an instance. There the recursion was missed, so in this model the program's meaning hits the guard at that call and is err: membership fails at the step where the bug was.

4 Well-foundedness

Theorem 5 (the model is well defined). For every program of the slice, and every choice of inaccessibles for its universe levels, there is exactly one signature meaning M and one meaning of terms and values satisfying the clauses of Section 3.

Proof sketch. By induction on the position k of the item; assume M for the items before k.

- **Terms and values.** These are finite trees: a term may embed values (a sealed program's arguments), and a value may embed code (a closure). Define their meanings by structural recursion on the tree. Every clause refers to one of three things:
  - the meanings of immediate subtrees, in an extended environment;
  - earlier items, through M;
  - inside a `fix`, the function being defined, at smaller rank, by the recursion of Definition 3, nested inside the structural one.

  No clause refers to the meaning of a type except through a subterm.
- **Item k.** A definition means its body read as a `fix` in the empty environment. An inductive means the least fixed point of an operator built from its field types' meanings, with D read as the variable family X; the operator is monotone by Lemma 1.

□

docs/18′s reason (Q2)	where it went
The meaning and R are defined in terms of each other.	One definition: a function type means its good functions (Section 2.1). $R_A = [\![A]\!]$.
R is undefined at a type variable.	A type variable means a set, and R at it is that whole set (Section 2.1).
A codomain is computed by running code, so it is not a subterm.	The codomain *term* is a subterm of the Π-term; a Π-value means its term under its captures (Section 2.2).
A type function recurses on data.	Its meaning is a function, defined once at its place in the program by guarded recursion on rank (Definition 3).
A type variable has no structure.	It is looked up in the environment, not recursed into.

Table 1: Where docs/18′s two reasons went.

5 The slice, worked

5.1 `Nat`

`Nat` is the inductive $\mathsf{Z} \mid \mathsf{S}(n : \mathsf{Nat})$. Its operator sends X to $\{(\ulcorner\mathsf{Z}\urcorner)\} \cup \{(\ulcorner\mathsf{S}\urcorner, e) \mid e \in X\}$, and its least fixed point is the set of numerals $\hat{0} = (\ulcorner\mathsf{Z}\urcorner)$, $\hat{1} = \left(\ulcorner\mathsf{S}\urcorner, \hat{0}\right)$, …, a copy of $\mathbb{N}$. We write $\hat{\mathbb{N}}$ for it.

5.2 A type function defined by recursion: `Cells`

From the suite (`Examples/CaseSplits.lean`):

```
inductive CellsEnd := End
inductive SliceOf (R : Type) := MkSlice(c : R)
inductive Cell (R : Type) := MkC(h : Nat, t : SliceOf(R))
def Cells (n : Nat) : Type by n := match n { Z => CellsEnd, S m => Cell(Cells(m)) }
```

- **The meaning of `Cells`.** `Cells` is a definition, so its meaning comes from Definition 3. At $\hat{0}$ it is the meaning of `CellsEnd`, $\{(\ulcorner\mathsf{End}\urcorner)\}$. At $(\ulcorner\mathsf{S}\urcorner, \hat{m})$ it is the meaning of `Cell` at the meaning of `Cells` at $\hat{m}$. That recursive call is under the guard, and it passes, since $\hat{m}$ is a component of $(\ulcorner\mathsf{S}\urcorner, \hat{m})$ and so has smaller rank.
- **The defining equation holds.** Lemma 4 applies, so $[\![\mathsf{Cells}]\!](\hat{n})$ is n nested cells ending in `End`.

- **A type that mentions it.** $\Pi(n : \mathsf{Nat})(c : \mathsf{Cells}(n))$. Nat has domain $\left\{(k, c) \mid k \in \hat{\mathbb{N}}, c \in [\![\mathsf{Cells}]\!](k)\right\}$ and means $\prod_{(k,c)\in \mathrm{Dom}} \hat{\mathbb{N}}$. Its meaning reads `Cells` only through M; nothing about `Cells`'s body is consulted again.
- **At the generic call.** The checker stores c's type as the sealed program $\lceil \mathsf{Cells}(\sigma_n) \rceil$. Under an assignment γ that means $[\![\mathsf{Cells}]\!](\gamma(\sigma_n))$, and nothing runs.
- **With an element type.** The array version `Cells(E, n)` (`Examples/ArrayBench.lean`) is the same with a type parameter E, which means $\eta(E)$.

5.3 A type variable, with R at it

Take $T = \Pi(A : \mathsf{Type}_0)(g : \Pi(y : \&A).\&A)(x : \&A).\&A$. For a set $X \in V_{\kappa_0}$, write $\mathrm{Good}(X)$ for the meaning of $\Pi(y : \&A).\&A$ with $A \mapsto X$:

$$\mathrm{Good}(X) = \left\{ G \in \prod_{d \in X} (X \times (X \to X)) \mid \mathrm{back}_G(d) \text{ is injective for every } d \right\}.$$

Then the domain of T is $\left\{(X, G, d) \mid X \in V_{\kappa_0}, G \in \mathrm{Good}(X), d \in X\right\}$. The meaning of T is the set of functions F on that domain with $F(X, G, d) \in X \times (X \to X)$ and $\mathrm{back}_F(X, G, d)$ injective.

- R at A is all of X. Nothing in T needs more of X's elements.
- T's class is "other" and its level is Type_1, because the parameter $A : \mathsf{Type}_0$ has sort Type_1 (Section 6). Its meaning is a member of V_{κ_1}, not of V_{κ_0}.
- Instantiating $A := \mathsf{Nat}$ gives $\mathrm{Good}\left(\hat{\mathbb{N}}\right)$, which is the meaning of $\Pi(y : \&\,\mathsf{Nat}).\&\,\mathsf{Nat}$.

The last point is the *substitution lemma*: the meaning of $T[A := V]$ in η equals the meaning of T in η extended with $A \mapsto$ the meaning of V. It is proved by induction on T:

- every clause reads A only through the environment;
- declared sorts are unchanged by substituting a type for a type variable of the same sort (D55).

No condition on the elements of X appears anywhere, which is Section 2.1′s point.

5.4 A function type returning a borrow

`TailM (x : &Nat) : &Nat` returns a borrow of the final `Z` inside `*x`. The meaning of its type is

$$\left\{ F \in \prod_{n \in \hat{\mathbb{N}}} \left(\hat{\mathbb{N}} \times \left(\hat{\mathbb{N}} \to \hat{\mathbb{N}}\right)\right) \;\middle|\; \mathrm{back}_F(n) \text{ is injective for every } n \right\}.$$

- `TailM`'s own meaning, $n \mapsto \left(\hat{0}, v \mapsto n + v\right)$, is in this set.
- A function that returns a leaked borrow, $n \mapsto \left(\hat{0}, v \mapsto n\right)$, is not, by construction.

docs/18′s `PF` shows why the second must be excluded:

```
def PF (x : &Nat) (e : Id(Unit, *x := 0, *x := 1)) : False := e
```

- **At its generic call.** `PF`'s hypothesis compares the two observations of the owner of `x`, which leave `0` and `1`. It computes to `Eq(Nat, 0, 1)`, which means $\emptyset$. So `PF`'s domain is empty, and its type means $\{\bullet\}$.
- **At a call through a good function.** Called with the borrow returned by some g in the set above, the hypothesis means $\mathrm{back}_g(n)\left(\hat{0}\right) = \mathrm{back}_g(n)\left(\hat{1}\right)$, which injectivity makes false. So no argument for `e` exists, and nothing follows.
- **If the set held the junk function.** The hypothesis would be true at that function, and `PF` would produce a member of $\emptyset$.

Stating the meaning of `Id` exactly needs the meaning of the state (Section 10). The point here is only that the type's meaning already excludes the junk function.

A function that *takes* such a function, $\Pi(g : \Pi(x : \&\,\mathsf{Nat}).\&\,\mathsf{Nat}).\,\mathsf{Nat}$, means $\prod_{G \in \mathrm{Good}\left(\hat{\mathbb{N}}\right)} \hat{\mathbb{N}}$. Its domain holds only good functions, so two of its elements that agree on every good argument are equal: this is [Conv-fun] (Section 2.1).

5.5 A Π into `Prop` over Type_0

- **A true one.** $\Pi(A : \mathsf{Type}_0)(x : A).\ \mathsf{Eq}(A, x, x)$ has class "returns proofs". It means $\{\bullet\}$ if the meaning of $\mathsf{Eq}(A, x, x)$ is $\{\bullet\}$ for every $X \in V_{\kappa_0}$ and every $d \in X$, which it is. So it means $\{\bullet\}$.
- **A false one.** $\Pi(A : \mathsf{Type}_0)(x : A)(y : A).\ \mathsf{Eq}(A, x, y)$ means $\emptyset$: take $X = \hat{\mathbb{N}}$, $\hat{0}$ and $\hat{1}$.
- **One over `Prop` itself.** $\Pi(P : \mathsf{Prop})(h : P).P$ means $\{\bullet\}$.

The clause quantifies over V_{κ_0}, which is a set, and its result is a truth value however large the domain. Impredicativity is harmless because the clause reads the codomain's meaning at each argument, never the meaning of the Π-type it is defining.

By contrast, the polymorphic identity's type $\Pi(A : \mathsf{Type}_0)(x : A).A$ has class "other" and level Type_1. It means $\prod_{(X,d)} X$, a member of V_{κ_1} but not of V_{κ_0}, as predicativity requires.

6 Sizes: where universe levels matter

Lemma 6 (sizes). Let T be a type term whose declared sort is Type_i, or `Prop`. Let η give every free variable of T a member of the meaning of its declared type. Then the meaning of T is err, or a member of V_{κ_i}, or, for `Prop`, a subset of $\{\bullet\}$.

Proof sketch. By induction on T. Since κ_i is inaccessible, V_{κ_i} is closed under finite tuples, subsets, and Σ and Π indexed by a member with members as fibres.

- An inductive's least fixed point lies inside V_{κ_0} by construction. (I6), that every field lives in `Prop` or Type_0, makes each field's meaning a subset of V_{κ_0} by induction, so the intersection in Φ removes nothing.
- A Π-type's level is the largest of its parameter types' sorts and its codomain's: `piSort` in `Machine.lean`, and `Decide.sortOf`, which the checker asserts agree. So every domain and fibre lies in V_{κ_i}, and so does the product. The one exception, a codomain in `Prop`, means a truth value.
- A level-j type with $j < i$ means a member of $V_{\kappa_j} \subseteq V_{\kappa_i}$. The model is cumulative although Ochr is not. This is harmless, because erasure is read from declared sorts, never from meanings.

□

This lemma is part of membership, not of the definition: no clause of Section 3 needed it.

A remark on RULES.md. P2 says "the sort of a Π-type is read from its codomain's declared sort". In context the sentence is about whether the sort is `Prop`, but read literally it would put $\Pi(A : \mathsf{Type}_0).A \to A$ in Type_0. That is impredicative Type_0, which has no set model. The checker takes the maximum with the parameters' sorts (`piSort`, `Decide.sortOf`), so only the wording is off; it should say so.

7 The Lean encoding

`ochr/core/lean/Ochr/Model/Slice.lean`, at commit `e90f0cf8` on main (it supersedes `2e950326` and `36eb0ecb`). It imports Lean core only, is not imported by `Ochr.lean`, and is checked with `lake env lean Ochr/Model/Slice.lean`. It builds with no `sorry`; its axioms are `propext`, and `Quot.sound`, which comes from function extensionality.

What it encodes.

- A type means a Lean type holding only its good elements, as in C1. $\Pi(x : \&T).\&U$ is the subtype of functions $d \mapsto (\mathrm{fwd}, \mathrm{back})$ whose backward function is injective (`Sem.brwRet`, `Sem.Back`).
- Type_0 means Lean's `Type`, every type of level 0, as V_{κ_0} does here. So there is no universe of codes and no induction–recursion.
- One interpretation function `I`, structurally recursive on a raw syntax of the slice, with earlier globals passed in as their meanings (induction on the program).
- Recursive type functions on `Nat` are defined by stages: the value at n is stage $n + 1$, and stage $k + 1$ reads the body with stage k answering the recursive call.

What it proves.

- `False` is empty (`false_empty`, `I_false_empty`).
- `tailM` is an element of `TailM`'s type, and that type excludes constant backward functions (`tailMTy_no_junk`).
- A type variable means its set (`I_tyvar`, by `rfl`).
- A Π into `Prop` over $\texttt{Type}_0$ is a `Prop` (`I_piP_type0`), and the polymorphic identity lives at level 1 (`I_poly_id_level1`).
- The meaning of $\Pi(g : \Pi(x : \& \texttt{Nat}).\& \texttt{Nat}).\ \texttt{Nat}$ (`I_takes_brwRet`).
- Extensionality for functions and for borrow-returning functions with no side condition (`pi_ext`, `brwRet_ext`), which is what [Conv-fun] needs.
- The semantic form of [Rec]. `Decreasing` says reading the body at n consults the recursive call only below n. Under it, `stage_stable` (stage k at m equals stage $m+1$ at m for every $k > m$) and `tfun_eq` (the defining equation at every n) hold. They are checked on `Cells` (a call at $n-1$: `cells_succ`, `pi_cells`) and on `Alt` (a call at $n-2$), where the file asserts that `rfl` fails, so the lemma is genuinely needed.
- The paired encoding's counterexample is kept as `Paired.agree_on_good` and `Paired.differ` (Section 2.1).

Where the paper and the encoding differ, and why.

- **Meaning and membership.** Lean is typed: every meaning must have a Lean type, so the meaning of a term that builds a borrow-returning function must be produced *together with* the proof that its backward function is injective. On paper, data is read untyped and functions as partial functions into $U \cup \{\text{err}\}$, so membership can be proved after the meaning is defined, in the same induction (docs/23, point 5). A mechanisation of test 2 must follow the Lean pattern: meaning and membership are built together.
- **Recursion.** Lean recurses by stages, on `Nat` only. The paper uses guarded recursion on set rank, for any inductive. Both define the meaning unconditionally and put [Rec] in a lemma about the defining equation (`Decreasing` with `tfun_eq`; Lemma 4). They differ only at a call that does not decrease: Lean answers with a lower stage, the paper with err.
- **Errors.** Lean reads an ill-formed type as the empty type, and an ill-formed numeral as 0; the paper keeps err distinct. Accepted programs never reach either, but a membership proof against the Lean encoding could succeed vacuously, for instance under a hypothesis whose ill-formed type reads as empty. A mechanised membership proof should keep an explicit error.
- **Environments.** Lean tags each environment entry with its syntactic sort, so that it knows the entry's Lean type; the paper's environments are untyped sets. This makes no difference to the meanings.
- **Levels.** Lean is bounded at levels 0 and 1 (docs/18 Q1, bounded option). The paper fixes $\kappa_0, ..., \kappa_N$ per program.
- **Coverage.** The Lean slice is narrower than the paper's (Section 9).

8 The choices

- **C1. A type means its good elements.** R is folded into the function-type clause, not paired in. Pairing is not needed at type variables (Section 2.1), and it would make conversion of functions unsound unless replaced by a partial equivalence relation (Lean: `Paired.differ`).
- **C2. Universes are all sets of their level**, V_{κ_i}, not only the meanings of Ochr types. So there is no universe of codes.
- **C3. Data is untyped, functions are typed by their annotations.** A constructor value means a tagged tuple whatever type it is used at. A closure means a function on its annotated domain, possibly partial (into $U \cup \{\text{err}\}$). In a typed mechanisation, meaning and membership are built together (Section 7).
- **C4. Proofs mean $\bullet$ by their declared sort**, never by their structure.
- **C5. The error value is strict**: it propagates unless a clause says otherwise.
- **C6. Recursive functions by guarded recursion on the set rank of the decreasing argument** (Lean: by stages, on `Nat`).
- **C7. Injectivity only.** A borrow-returning function's backward function must be injective, and need satisfy no lens law.

- **C8. Type-returning functions carry no final contents**, since their calls are erased.
- **C9. `J` and `rewrite` mean their transported term.** So docs/18′s question about a `J` stuck at endpoints that are equal but not convertible does not arise at the level of meaning: a closed-off `J` means the same as its arm.
- **C10. `Eq`'s meaning does not read its type slot.** This is what makes an equation field over the inductive being declared (Fording) monotone.
- **C11. Constructor values ignore their recorded parameters** (V2). Propositions are truncated after the fixed point.

9 What this test does not settle

- **The state, and so `Id`.** Borrows, loans waiting in fills, holes, inert loans, `⊥`, assignment, [End] and [Close] have no meaning yet, and `Id`'s meaning needs the meaning of imperative terms inside the meaning of types. On paper this adds clauses to the same recursion: types are terms, so term meanings and type meanings were always one structural recursion. `Id`'s sides are subterms, and the functions they call are earlier items, environment values, or closures that mean what their code means. So `Id` adds no new *kind* of reference, and the stratification of Theorem 5 still covers it. Two things keep it from being free, and they are the next hardest piece:
  - the *semantic state* those clauses run on must be defined: a state with live borrows as a function from the final contents of its holes, after Aeneas;
  - every type must be shown to read the state only through contents it can read when it is formed (Π-types capture no borrow, RULES P2; an annotation is formed on a private copy, after [Access] has ended the borrows it reads through), so that a type's meaning never depends on a hole.

  In Lean, `Id` needs the meanings of state transformers inside `Den`, a semantic store over all level-0 types: an encoding cost.
- **Membership.** No membership statement is proved here. It includes:
  - the injectivity of every accepted borrow-returning function (docs/18 (T4));
  - that the guard never fires (Lemma 4);
  - in Lean, a syntactic condition implying `Decreasing`, which needs a congruence lemma over all 17 term forms (routine).
- **Covered on paper but not in Lean:**
  - user inductives as least fixed points beyond `Pair` and `Unit`;
  - dependent constructors (telescopes and Fording);
  - type functions of arbitrary signature (Lean has $(A : \mathsf{Type}_0)(n : \mathsf{Nat})$, recursive on n);
  - `Eq` beyond `Nat`.
- **Universes in the uniform statement.** The model needs one inaccessible per level the program uses. The uniform statement over all programs needs one for every level, which is docs/18′s Q1, unchanged.
- **Outside the slice.** Inductives declared `abstract` or `implemented by` (the array library) need a clause of their own.

10 What test 2 inherits

- **The clauses of Section 3 and choices C1–C11.**
- **The interface for the state.** The meaning of a term becomes a *state transformer*: from an environment and a semantic state to a value and a semantic state, or err. The pure clauses are the case where the state is unchanged. Test 2 adds the clauses for borrowing, assignment, [End], [Close] and observation.
- **Function meanings with borrow parameters** are read off the body's state transformer:
  - Fin is the final contents of the parameter cells;

  - for a borrow result, fwd is the returned borrow's content at return, and back(v) is the final contents after the caller leaves v in it. This is Aeneas's backward function, and Ochr's [Close] fills with a hole.
- **Obligations, proved with the meaning (Lean) or in the same induction (paper):**
  - every borrow-returning closure the run builds has an injective backward function;
  - every closure that calls f calls it below the entry rank (Lemma 4);
  - [Conv-fun] is sound, which is function extensionality under C1;
  - `⊥` means "no information", as docs/18 now says.
- **For the higher-order slice:**
  - function parameters range over good sets (C1);
  - a closure built in the run and called later means a function on its annotated domain (C3);
  - a function that takes a function returning a borrow has been interpreted in Lean (`I_takes_brwRet`).

11 Summary for docs/18

1. Test 0 passes on the slice: on paper, and for most of the slice in Lean (`Ochr/Model/Slice.lean`, `e90f0cf8`: no `sorry`; axioms `propext` and `Quot.sound`).
2. Q2′s first circularity goes by letting a function type mean only its good functions, so R at every type is the meaning. Pairing R in is not needed at type variables, and would make conversion of functions unsound (Lean: `Paired.differ`).
3. Q2′s second circularity goes by defining meanings on terms, by recursion on program position, term structure and, inside a `fix`, the rank of the decreasing argument. Type_i means V_{κ_i} outright, so there is no universe of codes, and levels matter only for sizes.
4. A `fix` means a function, by guarded recursion, whether or not it passed [Rec]. So [Rec] moves from well-formedness to membership (the guard never fires), and an assignment's "in R" condition is redundant.
5. The next hardest piece is the meaning of the state (borrows, holes, `⊥`) and so of `Id` inside types. It adds clauses to the same recursion, and in a typed mechanisation meaning and membership must be built together.